

CottonGPT Project Documentation

Project Overview

ResearchGPT is a Python-based research assistant tool that leverages Large Language Models (LLMs) to process PDF documents, extract and clean data, generate embeddings, and provide an interactive question-answering interface for research papers. The system combines document understanding, semantic search, context-aware conversations, and external search capabilities to deliver comprehensive research assistance.

System Architecture

The project is organized with a modular architecture consisting of the following key components:

Core Components

- 1. PDF Processing (src/pdf_processor.py)**
 - Handles extraction of text and metadata from PDF files
 - Detects document domains for specialized processing
 - Provides validation and quality checks for PDF datasets
- 2. Data Cleaning (src/data_cleaner.py)**
 - Cleans and normalizes extracted text
 - Extracts metadata (title, authors, abstracts, etc.)
 - Identifies and segments document sections
 - Handles chunking of documents for embedding
- 3. Embeddings Management (src/embeddings_manager.py)**
 - Manages creation and storage of document embeddings
 - Provides semantic search capabilities
 - Integrates with LLMs for question answering
 - Handles document retrieval based on relevance
- 4. Context Management (src/context_manager.py)**
 - Maintains conversation history for context-aware responses
 - Manages state persistence between sessions
 - Builds combined context from history and retrieved documents
- 5. External Search (src/external_search.py)**
 - Supplements document knowledge with web search results
 - Integrates DuckDuckGo search API for up-to-date information

- Formats external search results for inclusion in answers

6. Utilities (src/utls.py)

- Provides logging functionality
- Manages configuration loading and saving
- Contains text processing utilities
- Provides model initialization helpers

7. Command Line Interface (src/cli.py)

- Provides user-friendly command interface
- Supports batch processing and interactive modes
- Handles configuration toggles for features

Data Flow

1. Input: PDF documents stored in data/raw/
2. Processing: PDFs are processed to extract text and metadata
3. Cleaning: Extracted text is cleaned and normalized
4. Storage: Processed documents are stored in data/processed/
5. Embedding: Document chunks are embedded and stored in data/embeddings/
6. Querying: User questions trigger document retrieval and LLM processing
7. Output: Responses are generated based on retrieved documents and context

File Structure

ResearchGPT/

- └─ config.yaml # Configuration settings
- └─ main.py # Main entry point for batch processing
- └─ update_embeddings.py # Script to update embeddings
- └─ README.md # Project documentation
- └─ requirements.txt # Python dependencies
- └─ src/ # Core source code
 - | └─ __init__.py # Package initialization
 - | └─ cli.py # Command line interface
 - | └─ context_manager.py # Conversation context management
 - | └─ data_cleaner.py # Text cleaning and metadata extraction
 - | └─ embeddings_manager.py # Vector embeddings and search
 - | └─ external_search.py # Web search integration
 - | └─ pdf_processor.py # PDF extraction and processing
 - | └─ utils.py # Utility functions and helpers
- └─ data/ # Data storage
 - | └─ raw/ # Raw PDF files
 - | └─ processed/ # Processed documents
 - | | └─ processed_papers.json # Initial processed documents
 - | | └─ results_papers.json # Enhanced documents with summaries
 - | └─ embeddings/ # Vector store for embeddings
- └─ logs/ # Log files
- └─ tests/ # Test files

Configuration System

Controlled by config.yaml:

- Processing Settings: Batch size, worker count
- Cleaning Settings: Text normalization options
- Output Settings: File formats and compression
- Logging Settings: Log levels and file paths
- Model Settings: Embedding and completion model selection
- Embeddings Settings: Model type and parameters
- Chunking Settings: Chunk size and overlap
- Retrieval Settings: Number of documents to retrieve
- LLM Settings: Model name and temperature
- External Search Settings: Engine and result count

Key Features

1. PDF Processing

- Bulk document processing
- Metadata extraction (title, authors, abstract)
- Section segmentation

2. Semantic Search

- Vector-based retrieval
- Relevance ranking
- Context-aware lookup

3. Interactive Q&A

- Command-line chat
- Follow-up question support
- Maintains conversation history

4. External Knowledge Integration

- DuckDuckGo-based search
- Combined with document responses
- Configurable on/off

5. Context Awareness

- Session memory
- Persistent history

- Clear context feature

Command Line Interface

- **process_pdfs**

```
python -m src.cli process_pdfs --config config.yaml --input-dir data/raw
```

- **process**

```
python -m src.cli process --config config.yaml
```

- **query**

```
python -m src.cli query --config config.yaml "What is the latest research on cotton diseases?"
```

- **start**

```
python -m src.cli start --config config.yaml
```

- **context**

```
python -m src.cli context --config config.yaml --enable
```

- **external_search**

```
python -m src.cli external_search --config config.yaml --enable --engine duckduckgo
```

- **web_search**

```
python -m src.cli web_search --config config.yaml "recent advances in cotton disease management"
```

Model Integration

- **Embedding Models:** Hugging Face (e.g., BAAI/bge-large-en-v1.5), OpenAI (optional)
- **Completion Models:** Hugging Face (e.g., microsoft/phi-2), OpenAI
- **Domain-Specific Models:** Optional via domain_models in config

Implementation Details

Embeddings Generation

1. Text extraction
2. Cleaning

3. Chunking
4. Embedding
5. Storage (Chroma)

Question Answering

1. User query
2. Document retrieval
3. Context building (from history + docs)
4. External search (optional)
5. Answer generation
6. History update

Context Management

- Stores history
- Persists session state
- Builds combined context
- Allows manual reset

Getting Started for New Developers

1. Environment Setup
 - `pip install -r requirements.txt`
2. Configuration
 - Update config.yaml as needed
 - Add API tokens if using external models
3. Data Prep
 - Place PDFs in data/raw/
4. Initial Processing
 - `python -m src.cli process`
5. Interactive Chat
 - `python -m src.cli start`

Key Dependencies

- PyPDF2
- sentence-transformers

- langchain
- Chroma
- transformers
- requests
- click
- nltk
- hugging face token

Extension Points

1. Add new search engines in `external_search.py`
2. Add support for new LLMs in `utils.py`
3. Add custom embedding models in config
4. Add domain-specific models

Troubleshooting

- **PDF Errors:** Check for encryption/corruption
- **Embedding Failures:** Verify model config & disk space
- **Slow Queries:** Tune chunking and switch to lighter models

Future Development

- Multi-modal inputs (images, graphs)
- More search engines
- Web UI or desktop app
- Shared sessions / multi-user support
- Export conversation/docs feature